

Software Engineer- Internship Work Assignment.

Productivity Tool- Team standup logger

Build a lightweight internal tool where team members can post their daily standup updates — what they worked on, what they're doing today, and any blockers — with a live activity feed and a productivity dashboard.

THE SCENARIO

Your team uses a daily standup meeting to sync up. You've been asked to build an internal web tool to replace it — team members post their standup updates asynchronously, see what everyone else posted in real time, and managers get a quick dashboard showing team health and blocker trends.

KEY FEATURES TO BUILD

1. Standup post form

3 fields: yesterday, today, blockers. Attach a file or screenshot.

2. Live activity feed

New posts appear without a page refresh via polling or WebSocket.

3. Productivity dashboard

Charts showing posts per day, blocker frequency, and team activity.

Weather + time API

Show each post's local time and weather using a public API.

DELIVERABLES

1. Backend — Python/Django/Flask

- StandupPost model: author (name), yesterday, today, blockers, has_blocker (bool), file attachment, timestamp.
- REST endpoints: GET /standups/, POST /standups/, GET /standups/stats/
- A /standups/stats/ endpoint returning posts per day and blocker count for the last 7 days.

2. Frontend — React

- Standup submission form with all three fields and a blocker checkbox.
- Feed that polls GET /standups/ every 10 seconds — no page refresh needed.
- Each post card shows author, timestamp, the three fields, and a blocker badge when flagged.
- A single bar chart (posts per day, last 7 days) using data from /standups/stats/.
- Current weather (temperature + condition) fetched from Open-Meteo and shown in the feed header.
- Basic error states: form errors shown inline, API failures shown to the user.

3. Third-party API integration

- Use Open-Meteo (free, no API key) to fetch current weather for a fixed city
- Display a weather icon and temperature on the feed page.
- Handle API errors gracefully — if the call fails, show a fallback rather than breaking the page.

4. Documentation

- README.md with setup steps for both backend and frontend.

- Screenshot of the dashboard included in the README.
- Deploy the application and share the link.

Kindly submit your work by **5th June 2026**. All the best!